

Assignment: Tomasulo Algorithm Implementation

In this assignment, you will start from existing code and implement the Tomasulo algorithm with support of issuing multiple instructions (1-4 instructions – in order). You will work with code that already implements the Tomasulo algorithm (single issue with details below). The Tomasulo algorithm supports register renaming, ROB, speculative execution, and memory disambiguation technique.

Processor configuration

The machine has 5 pipeline stages: **ISSUE, EX, MEM, WB, and COMMIT**. The **ISSUE stage** also includes instruction fetch and decode. The **EX stage** for loads and stores calculates their effective address (as explained below). Branch resolution is also done in the EX stage. The tasks performed in each stage are in line with what we discussed in class except for the specifications given for the load/store queue and branch unit.

The main components of the processor are: one instruction buffer, architecture register files (integer and floating point), register aliasing table (RAT), reservation stations for each function units, one CDB, one ROB, one branch predictor including the target buffer, and one load/store queue. The ROB fields are defined as explained in class. The details of the load/store queue are given below.

Load/Store queue. All loads and stores are enqueued in their program order. The queue can be viewed as the reservation stations for the memory unit. There is a dedicated adder for effective address calculation. This is done in the EX stage of the load/store instruction. Hence, loads and stores do not occupy the integer ALU. Each entry in the queue contains two fields: address and value (not useful for loads).

Stores write into the memory only in the commit stage. Hence, they are dequeued only when they commit. A load, however, can go to memory whenever the address is calculated, and no forwarding-from-a-store was found. Forwarding-from-a-store is checked in the memory stage. It takes 1 cycle to perform the forwarding if a match is found. If not found, or there are still unresolved memory addresses for stores, then the load accesses the memory for data. Once a load returns from the memory or gets the value from a previous store, its entry in the load/store queue is cleared. Note that it is correct not to clear this entry, but the queue can quickly fill up, causing structure hazards for future loads/stores.

The details about the branch unit:

Branch unit. Branch instructions are issued into the reservation stations of an integer ALU. The code implements a simple one-bit predictor for each branch instruction. Use a BTB of 8 entries to store the target. Use the least significant 3 bits of the word address of the PC to index into the BTB. Prediction is done in the first cycle of execution (ISSUE stage). The branch is resolved at the end of the EX stage. Upon a misprediction, actions must be taken to squash wrong instructions.

These include:

1. Recover the RAT;
2. Clear the reservation station's wait-for tag fields for wrong instructions;

3. Clear ROB entries that surpass the branch. Assume these actions take one cycle, and fetching from the correct instruction starts in the next cycle. For example, if misprediction is detected in cycle n , correct fetch should start at cycle $n+2$.

Other hardware units include:

Integer adder; unpipelined

FP adder; pipelined

FP multiplier; pipelined

Integer and FP register files, 32 entries each. Integer R0 is hardwired to 0.

Memory; single-ported; non-pipelined

The hardware configuration is fully parameterizable, e.g. the # of ROB entries, the # of reservation stations for each FU, the # of execution cycles for each FU, the # of cycles for memory access etc. The model simulates a “memory” of 256B (64W) which stores the data for a running testbench. In this exercise, you need to add multi-issue support (in-order) to the simulator.

Instruction Set Architecture

Data Transfer Instructions

Ld Fa, offset(Ra)	Load a single precision floating point value to Fa
Sd Fa, offset(Ra)	Store a single precision floating point value to memory

Control Transfer Instructions

Beq Rs, Rt, offset	If $R_s == R_t$ then branch to $PC+4+offset \ll 2$
Bne Rs, Rt, offset	If $R_s \neq R_t$ then branch to $PC+4+offset \ll 2$

ALU Instructions

Add Rd, Rs, Rt	$R_d = R_s + R_t$	Integer
Add.d Fd, Fs, Ft	$F_d = F_s + F_t$	FP
Addi Rt, Rs, immediate	$R_t = R_s + \text{immediate}$	Integer
Sub Rd, Rs, Rt	$R_d = R_s - R_t$	Integer
Sub.d Fd, Fs, Ft	$F_d = F_s - F_t$	FP
Mult.d Fd, Fs, Ft	$F_d = F_s * F_t$, assuming that Fd is enough to hold the result	FP

Inputs

The input to the simulator is a text file containing assembly instructions specified in the above format. The text is case insensitive. The configurations of the hardware units such as RS numbers and EX cycles are also provided through this input file. The registers are initialized to zero unless specified in the input.

Memory addresses are byte addresses. A sample input is shown below.

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	2	1		1
FP adder	3	3		1
FP multiplier	2	20		1
Load/store unit	3	1	4	1

You need to add support for specifying issue width (1 to 4 – default is 1).

ROB entries = 128

Issue width = 2

R1=10, R2=20, F2=30.1

Mem[4]=1, Mem[8]=2, Mem[12]=3.4

Add.d F1, F2, F3

Ld F4, 8(R1)

Bne R2, R3, -3

Outputs

The output from the simulator contains information about the final instruction status table including the cycles in each stage, and the results of the executed program including both registers and memory contents (non-zeros), formatted as the following.

	ISSUE	EX	MEM	WB	COMMIT
Instruction_1					
Instruction_2					
Instruction_3					
...					
REGISTER VALUES					
	R0	R1	R2...		
value:					
	F0	F1	F2...		
value:					

NON-ZERO MEMORY VALUES

Addresses	Values
addr_1	xxx
addr_2	yyy
addr_3	zzz
...	

Test benchmarks

In addition to test cases that are provided, you can write your own test cases. This part should be included in your report. Your test cases should be able to test correctness of your implementation to support different issue width.

Report

Submit your code, test cases, a simple readme file, and report to github.

In your report, include the following section:

- Brief description what you have added to the code
- Show results of test cases under different issue width

Make sure that your code can compile and execute. If your program has bugs that have not been fixed, please in your report, explain what works and what does not work.